

Introduction to EidosLang

A language for expressing mathematical theories

What Is Eidos?

EidosLang is a language for expressing mathematical theories of all kinds. It is designed to be a unified framework that naturally encompasses:

- Propositional logic (including quantified propositional logic)
- Equational logic
- First-order logic (FOL)
- Second-order logic (SOL)
- Classical and intuitionistic logic
- Many-sorted and single-sorted logic

Eidos is based on the discovery that **Quantified Intuitionistic Propositional Logic (QIPL)** has the same expressive power as first-order logic — it is Turing-complete — while being remarkably simple. Eidos can be thought of as syntactic sugar on top of QIPL.

Unlike proof assistants such as Coq or Lean, Eidos does *not* support verifying mathematical proofs. Its sole goal is to *express* mathematical objects and theories in a simple, natural way. A significant part of the Lean 4 export implementation is already complete; Coq export is also planned.

The Name

The Greek word εἶδος ('form'/'shape'/'appearance') comes from Plato, who used it for the abstract 'ideas'/'forms' studied in mathematics — e.g., the idea of a circle or a triangle. There is also a connotation with Husserl's philosophy ('eidetic').

The Mereological Foundation

Eidos is built on a **mereological** (part-whole) interpretation of logic. The implication $P \rightarrow Q$ is read as "P includes Q as a part" (Q is part of P). This gives rise to three core operations present in every Eidos theory:

- $P + Q$ (conjunction / mereological sum) — the smallest thing that includes all parts of both P and Q
 - $P \times Q$ (disjunction / mereological product) — the largest thing included as a part in both P and Q
 - $Q - P$ (implication / mereological difference) — what needs to be added to P to get Q
-

File Extensions

Eidos uses **.theory** as its general file extension. More specific extensions constrain which sublanguage features are available:

Extension	Sublanguage
.eq.theory	Equational logic
.reg.theory	Regular logic
.coh.theory	Coherent logic
.fol.theory	First-order logic
.sol.theory	Second-order logic
.prop.theory	Propositional logic
.mereo.theory	Pure mereological theories

Basic Structure of a Theory

Every .theory file defines a theory enclosed in { }. A theory may contain up to three sections, separated by commas:

```
{
  signature {
    // Sorts, functions, relations, individuals, sets,
    // propositional constants
  },
  axioms {
    // facts, assertions, and/or metafacts
  },
  subtheories {
    // named, implicit, or reflection subtheories
  }
}
```

Sorts

Sorts are the “types” of Eidos. Sort names must start with an uppercase letter.

```
sort MySort;           // declare a sort
MyOtherSort subsort MySort; // subsort (more specific)
MyQuotientSort quotient MySort; // quotient sort (coarser)
MySubQuotient subquotient MySort; // combination of both
```

Subsort: An individual of the parent sort may fail to be self-identical under the subsort’s identity relation — the subsort can “collapse” some elements to nothing.

Quotient sort: Two distinct individuals of the parent sort may be identified as the same individual under the quotient sort’s identity relation.

Built-in Sorts

Three special sorts are available in every theory:

- $\mathbb{D}$ — “the domain”; used for single-sorted theories or as the primary sort
- $\mathbb{U}$ — “the universe”; includes everything in the theory (only usable in metafacts)
- $\mathbb{P}$ — the sort for propositions; elements can be combined with logical connectives

Individuals and Sets

Individual names start with a **lowercase** letter; set names start with an **uppercase** letter.

```
myObject : S;    // individual of sort S (lowercase name)
MySet  $\subseteq$  S;    // set of sort S (uppercase name)
```

Example — declaring a sort, an individual, a set, and an axiom:

```
{
  signature {
    sort MySort;
    myIndividual : MySort;
    MySet  $\subseteq$  MySort;
  },
  axioms {
    assertions {
      myIndividual  $\in$  MySet;
    }
  }
}
```

There is no distinction between sets and monadic predicates. These two lines mean exactly the same thing:

```
i  $\in$  MySet
MySet(i)
```

Set operations correspond directly to logical connectives:

```
(S  $\cap$  T)(a)  $\leftrightarrow$  S(a)  $\wedge$  T(a)
(S  $\cup$  T)(a)  $\leftrightarrow$  S(a)  $\vee$  T(a)
```

Functions and Relations

First-Order Functions — lowercase names

```
f : S, T  $\rightarrow$  U;    // 2-ary function S  $\times$  T  $\rightarrow$  U
inv :  $\mathbb{D} \rightarrow \mathbb{D}$ ;    // 1-ary function on  $\mathbb{D}$ 
```

First-Order Relations — uppercase names. $\subseteq$ notation

```
R  $\subseteq$  S, S;    // binary relation on S
```

Second-Order Functions/Relations — uppercase names

Second-order functions operate on **sets** rather than individuals. A second-order relation is a second-order function whose result sort is $\mathbb{P}$.

```
F : S, T → U;      // 2-ary SOL function
XOR : P, P → P;    // SOL relation
```

The distinction between FOL and SOL is determined by capitalisation: lowercase = FOL, uppercase = SOL. Sets, relations, and propositional constants all start with uppercase.

Axioms: Facts, Assertions, Metafacts

Facts — enforced equational logic

Facts correspond to **positive/equational logic**. They are *enforced* — the compiler treats them as definitional identifications.

- Can use equations (=)
- Can use mereological sums (+) and products (×)
- **Cannot** use propositional connectives ($\neg$, $\rightarrow$, $\vee$, $\wedge$) or the sort $\mathbb{P}$
- **Cannot** use inequalities (an inequality is a negated equality, requiring $\neg$)

```
{
  signature {
    n : D;
    op : D, D → D;
  },
  axioms {
    facts {
      ∀x:D ∀y:D ∀z:D op(x,op(y,z)) =_D op(op(x,y),z);
      ∀x:D op(n,x) =_D x;
      ∀x:D op(x,n) =_D x;
    }
  }
}
```

Assertions — full logic

Assertions are **full first-/second-order logic** statements. They are *not enforced* — they hold conditionally on consistency. An assertion “a =_S b” means “a and b are identical, *or there is a contradiction*”.

- Can use all propositional connectives: $\neg$, $\rightarrow$, $\leftarrow$, $\leftrightarrow$, $\wedge$, $\vee$, $\top$, $\perp$
- Can use quantifiers: $\forall$, $\exists$

```
{
  signature { sort S; R ⊆ S, S; },
  axioms {
    assertions {
      ∀x:S ∀y:S R(x,y) ↔ R(y,x); // R is symmetric
    }
  }
}
```

Metafacts — maximum expressive power

Metafacts are for special situations where facts and assertions are insufficient. They work at a finer granularity than assertions.

- Use $\Rightarrow$ instead of $\rightarrow$ for implication ($P \rightarrow P = \top$, but $P \Rightarrow P = \mathbb{U} \# \text{min}$)
- Use $+$ and $\times$ instead of $\wedge$ and $\vee$
- The “true” value is $\mathbb{U} \# \text{min}$ (fine-grained truth), not $\top$
- The sort $\mathbb{U}$ can only be used in metafacts

Quantifiers

```
∀x:S x ∈ MySet      // quantify over individuals
∃X⊆S ∀Y⊆S X =_S Y  // quantify over sets
∀X:ℙ (X ∨ ¬X)       // quantify over propositions (LEM)
```

Subtheories

A theory can import subtheories in three ways. Subtheories may be defined inline or loaded from external files using `@filename`.

Named subtheories

The subtheory name **must always** be used explicitly when referring to its contents.

```
subtheories {
  named {
    sub1: { signature { sort S; f : S → S; } }
  }
}
// Access as: sub1.S, sub1.f (cannot abbreviate)
```

Implicit subtheories

The subtheory name is **optional** — contents can be used without qualification. If the same name appears in multiple implicit subtheories it is assumed to be the same thing (a compilation error is generated if types are incompatible).

```
subtheories {
  implicit {
    sub3: { signature { sort U; } }
  }
}
// Access as: U (sub3.U also works)
```

Reflection subtheories

Reflection transforms a theory’s contents systematically — analogous to the powerset construction in topos theory:

- Sorts become “reflected sorts” (powerset sorts — subsets become individuals)
- SOL functions become FOL functions
- FOL functions remain FOL but with reflected argument sorts
- Mereological objects become individuals

- The part-whole relation becomes an order relation

Reflection cannot be combined with implicit import. Everything imported by reflection must be prefixed with the subtheory name.

```
subtheories {
  reflection {
    sub: {
      signature {
        sort S; i : S; Pred  $\subseteq$  S; F: S, S  $\rightarrow$  S;
      }
    }
  }
}
// sub.S -- sort whose individuals are subsets of the original S
// sub.F -- now a FOL function (was SOL)
// sub.Pred -- now an individual (was a set)
```

Propositional Logic

```
{
  signature { P :  $\mathbb{P}$ ; Q :  $\mathbb{P}$ ; R :  $\mathbb{P}$ ; },
  axioms {
    assertions {
      P  $\rightarrow$  (Q  $\wedge$  R);    // if P then Q and R
      Q  $\rightarrow$  (P  $\vee$   $\neg$ R);  // if Q then P or not-R
    }
  }
}
```

Available connectives: $\neg$, $\rightarrow$, $\leftarrow$, $\leftrightarrow$ (= for propositions), $\wedge$, $\vee$, $\top$, $\perp$. All laws of **intuitionistic logic** are guaranteed. Classical logic (Law of Excluded Middle) is not assumed by default but can be asserted:

```
 $\forall X:\mathbb{P} (X \vee \neg X)$ 
```

Sorted Identity Relations

For a sort S:

- $=_S$ — identity among *individuals* of sort S
- $=^S$ — identity among *sets* of sort S (equal iff same elements)

Quick Reference: Naming Conventions

What	Convention	Examples
Sorts	Uppercase	MySort, $\mathbb{D}$, $\mathbb{P}$, $\mathbb{U}$
Individuals	Lowercase	myObject, x, n
FOL functions	Lowercase	f, op, inv

Sets	Uppercase	MySet, $S \# \max$
FOL/SOL relations	Uppercase + $\subseteq$	$R \subseteq S$, S
SOL functions	Uppercase	$F : S \rightarrow T$
Propositional constants	Uppercase, sort $\mathbb{P}$	$P : \mathbb{P}$

Complete Examples

Monoid

```
{
  signature {
    n :  $\mathbb{D}$ ;
    op :  $\mathbb{D}, \mathbb{D} \rightarrow \mathbb{D}$ ;
  },
  axioms {
    facts {
       $\forall x:\mathbb{D} \forall y:\mathbb{D} \forall z:\mathbb{D} \text{ op}(x, \text{op}(y,z)) = \text{op}(\text{op}(x,y), z)$ ;
       $\forall x:\mathbb{D} \text{ op}(n, x) = x$ ;
       $\forall x:\mathbb{D} \text{ op}(x, n) = x$ ;
    }
  }
}
```

XOR as a second-order relation

```
{
  signature {
    XOR :  $\mathbb{P}, \mathbb{P} \rightarrow \mathbb{P}$ ;
  },
  axioms {
    assertions {
       $\forall P:\mathbb{P} \forall Q:\mathbb{P} \text{ XOR}(P,Q) \leftrightarrow ((\neg P \wedge Q) \vee (P \wedge \neg Q))$ ;
    }
  }
}
```

Symmetric relation

```
{
  signature {
    sort S;
     $R \subseteq S, S$ ;
  },
  axioms {
    assertions {
       $\forall x:S \forall y:S R(x,y) \leftrightarrow R(y,x)$ ;
    }
  }
}
```

Subsorts and quotients

```
{
  signature {
    sort S;
    T subsort S;
    a : S;
  },
  axioms {
    assertions {
      ¬(a =_T a); // a is not self-identical in T (collapsed)
      a =_S a; // but is self-identical in S
    }
  }
}
```